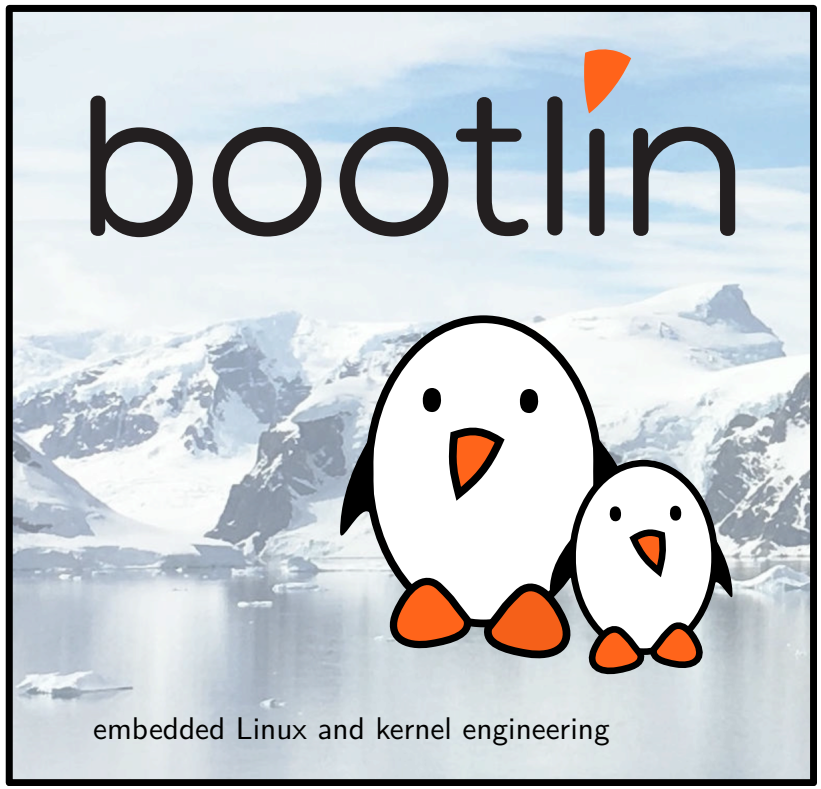
Linux kernel and driver development training



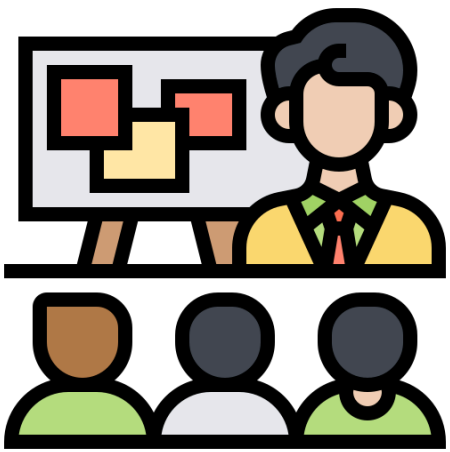
© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Latest update: November 08, 2026.

Document updates and training details:
https://bootlin.com/training/linux-kernel

Corrections, suggestions, contributions and translations are welcome!
Send them to feedback@bootlin.com

Linux kernel and driver development training

- These slides are the training materials for Bootlin's *Linux kernel and driver development* training course.
- If you are interested in following this course with an experienced Bootlin trainer, we offer:
  - **Public online sessions**, opened to individual registration. Dates announced on our site, registration directly online.
  - **Dedicated online sessions**, organized for a team of engineers from the same company at a date/time chosen by our customer.
  - **Dedicated on-site sessions**, organized for a team of engineers from the same company, we send a Bootlin trainer on-site to deliver the training.
- Details and registrations: https://bootlin.com/training/linux-kernel
- Contact: `training@bootlin.com`



Icon by Eucalyp, Flaticon

About Bootlin

© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!





Bootlin introduction

- Engineering company
  - In business since 2004
  - Before 2018: *Free Electrons*
- Team based in France and Italy
- Serving **customers worldwide**
- **Highly focused and recognized expertise**
  - Embedded Linux
  - Linux kernel
  - Embedded Linux build systems
- **Strong open-source** contributor
- Activities
  - **Engineering** services
  - **Training** courses
- https://bootlin.com

bootlin



Bootlin engineering services

Bootloader / firmware development

U-Boot, Barebox, OP-TEE, TF-A, .../

Linux kernel porting and driver development

Linux BSP development, maintenance and upgrade

Embedded Linux build systems

Yocto, OpenEmbedded, Buildroot, ...

Embedded Linux integration

Boot time, real-time, security, multimedia, networking

Open-source upstreaming

Get code integrated in upstream Linux, U-Boot, Yocto, Buildroot, ...



Bootlin training courses

Embedded Linux system development On-site: 4 or 5 days Online: 7 * 4 hours	Linux kernel driver development On-site: 5 days Online: 7 * 4 hours	Yocto Project system development On-site: 3 days Online: 4 * 4 hours	Buildroot system development On-site: 3 days Online: 5 * 4 hours	Embedded Linux networking On-site: 3 days Online: 4 * 4 hours
Understanding the Linux graphics stack On-site: 2 days Online: 4 * 4 hours	Embedded Linux audio On-site: 2 days Online: 4 * 4 hours	Embedded Linux Security On-site: 3 days Online: 4 * 4 hours	Linux debugging, tracing, profiling and performance analysis On-site: 3 days Online: 4 * 4 hours	Real-Time Linux with PREEMPT_RT On-site: 2 days Online: 3 * 4 hours

All our training materials are freely available
under a free documentation license (CC-BY-SA 3.0)
See https://bootlin.com/training/



Bootlin, an open-source contributor

- Strong contributor to the **Linux** kernel
  - In the top 30 of companies contributing to Linux worldwide
  - Contributions in most areas related to hardware support
  - Several engineers maintainers of subsystems/platforms
  - 9000 patches contributed
  - https://bootlin.com/community/contributions/kernel-contributions/
- Contributor to **Yocto Project**
  - Maintainer of the official documentation
  - Core participant to the QA effort
- Contributor to **Buildroot**
  - Co-maintainer
  - 6000 patches contributed
- Significant contributions to U-Boot, OP-TEE, Barebox, etc.
- Fully **open-source training materials**

Bootlin on-line resources

- Website with a technical blog:
  https://bootlin.com
- Engineering services:
  https://bootlin.com/engineering
- Training services:
  https://bootlin.com/training
- LinkedIn:
  https://www.linkedin.com/company/bootlin
- Elixir - browse Linux kernel sources on-line:
  https://elixir.bootlin.com



Icon by Freepik, Flaticon

Generic course information



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!

Beaglebone Black / Beaglebone black wireless shopping list

- BeagleBone Black or BeagleBone Black Wireless, from BeagleBoard.org
  - Texas Instruments AM335x (ARM Cortex-A8 CPU)
  - 512 MB of RAM
  - 4 GB of on-board eMMC storage
  - Plenty of peripherals and features
  - 2 x 46 pins headers, with access to many expansion buses (I2C, SPI, UART and more)
- MicroUSB cable
- USB Serial Cable - 3.3 V - Female ends (for serial console)[1]
- Nintendo Nunchuk with UEXT connector[2]
- Breadboard jumper wires - Male ends (to connect the Nunchuk)[3]
- MicroSD card









[1] https://www.olimex.com/Products/USB-Modules/Interfaces/USB-SERIAL-F

[2] https://www.olimex.com/Products/Modules/Sensors/MOD-WII/MOD-Wii-UEXT-NUNCHUCK/

[3] https://www.olimex.com/Products/Breadboarding/JUMPER-WIRES/JW-110x10/

BeaglePlay shopping list

- BeaglePlay, from BeagleBoard.org
  - Texas Instruments AM625x (4×ARM Cortex-A53 CPU)
  - 2 GB RAM
  - 16 GB of on-board eMMC storage
  - Peripherals: SPI, I2C, UART, USB...
- USB-C cable for the power supply
- A USB-FTDI cable
- RJ45 cable for networking
- A micro SD card with at least 2G of capacity
- Nintendo Nunchuk with UEXT connector[1]
- Breadboard jumper wires - Male ends (to connect the Nunchuk)[2]







[1] https://www.olimex.com/Products/Modules/Sensors/MOD-WII/MOD-Wii-UEXT-NUNCHUCK/

[2] https://www.olimex.com/Products/Breadboarding/JUMPER-WIRES/JW-110x10/

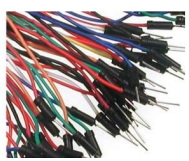
IMX93 FRDM shopping list

- NXP i.MX93 11x11 FRDM board (76 EUR + VAT on Mouser)
  - NXP i.MX93 (Dual ARM Cortex-A55 + Cortex-M33)
  - 2 GB LPDDR4
  - 32 GB of on-board eMMC storage
  - Plenty of peripherals: I2C, SPI, UART, USB...
- 2 USB-C cable for the power supply and the serial console
- RJ45 cable for networking
- MicroSD card
- USB Serial Cable - 3.3 V - Female ends[1]
- Nintendo Nunchuk with UEXT connector[2]
- Breadboard jumper wires - Male/Female ends (to connect to Nunchuk)[3]









[1] https://www.olimex.com/Products/USB-Modules/Interfaces/USB-SERIAL-F

[2] https://www.olimex.com/Products/Modules/Sensors/MOD-WII/MOD-Wii-UEXT-NUNCHUCK/

[3] https://www.olimex.com/Products/Breadboarding/JUMPER-WIRES/JW-200x10-FM/



Training quiz and certificate

- To get your training certificate you must
  1. Attend all sessions of this training course
  2. Achieve more than 50% of correct answers at our final quiz
     - The final quiz questions are identical to the pre-training quiz
     - The final quiz must be completed within two weeks of the session end's date
- The training certificate will be sent to you two weeks after the session end's date.



Participate!

During the lectures...

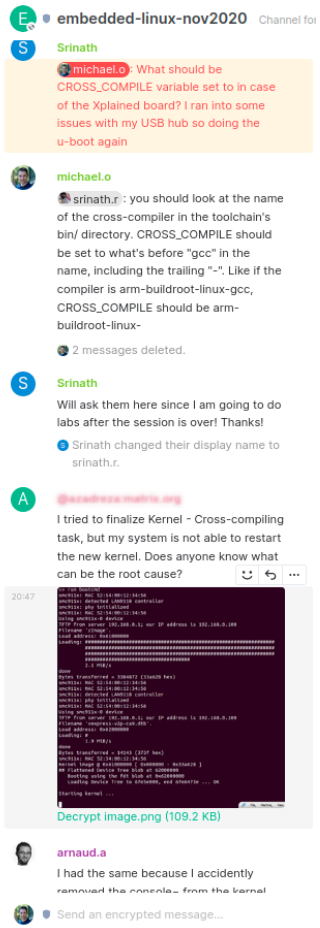
- Don't hesitate to ask questions. Other people in the audience may have similar questions too.
- Don't hesitate to share your experience too, for example to compare Linux with other operating systems you know.
- Your point of view is most valuable, because it can be similar to your colleagues' and different from the trainer's.
- In on-line sessions
  - Please always keep your camera on!
  - Also make sure your name is properly filled.
  - You can also use the "Raise your hand" button when you wish to ask a question but don't want to interrupt.
- All this helps the trainer to engage with participants, see when something needs clarifying and make the session more interactive, enjoyable and useful for everyone.



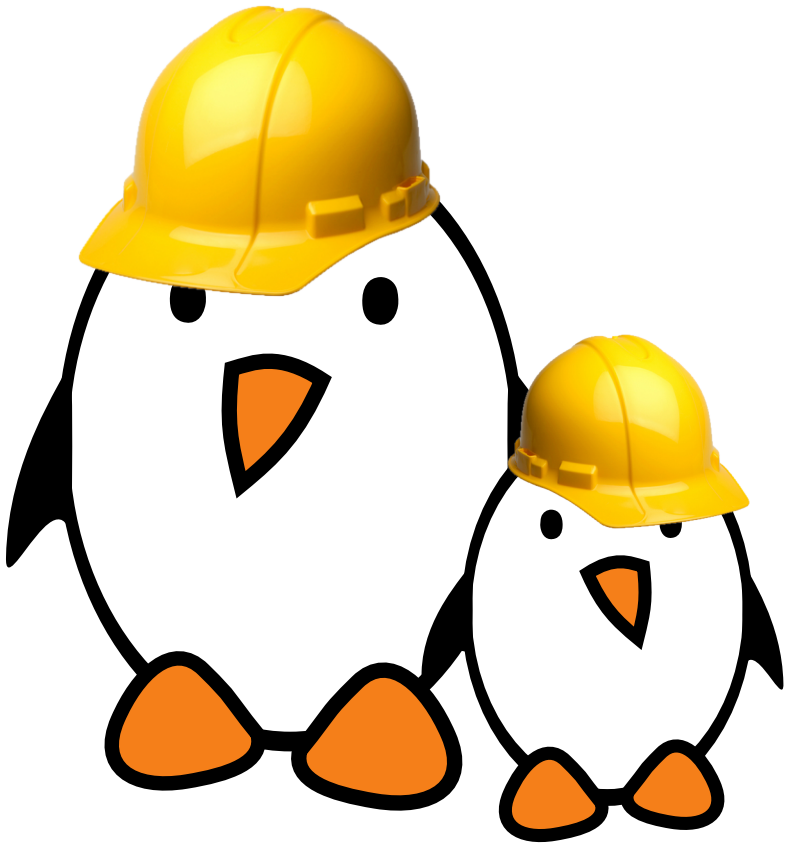
Collaborate!

As in the Free Software and Open Source community, collaboration between participants is valuable in this training session:

- Use the dedicated Matrix channel for this session to add questions.
- If your session offers practical labs, you can also report issues, share screenshots and command output there.
- Don't hesitate to share your own answers and to help others especially when the trainer is unavailable.
- The Matrix channel is also a good place to ask questions outside of training hours, and after the course is over.



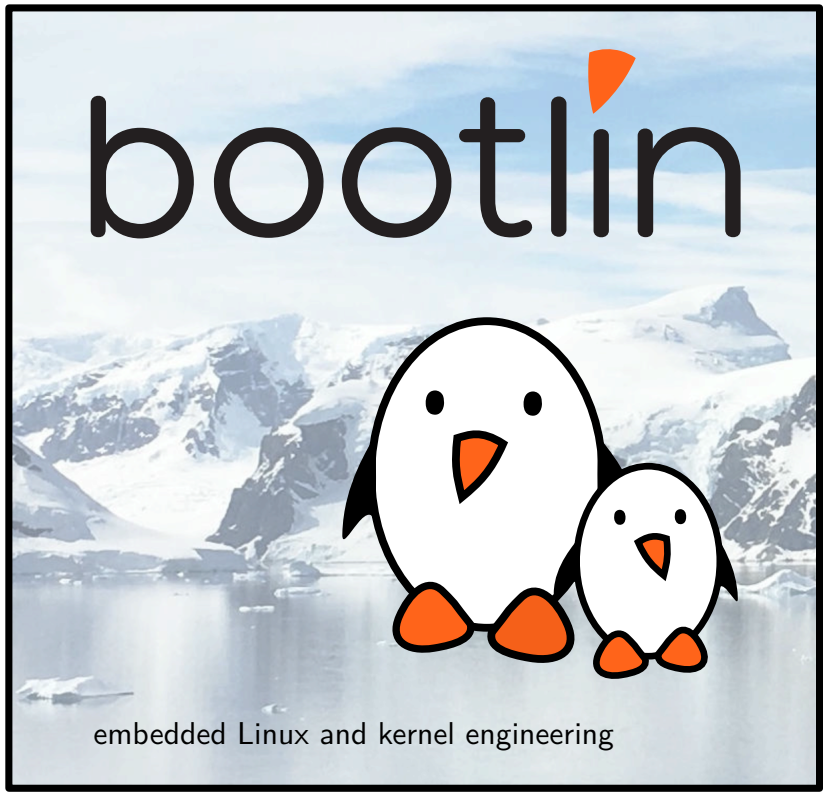
Practical lab - Training Setup



Prepare your lab environment

- Download and extract the lab archive

Linux Kernel Introduction



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Origin

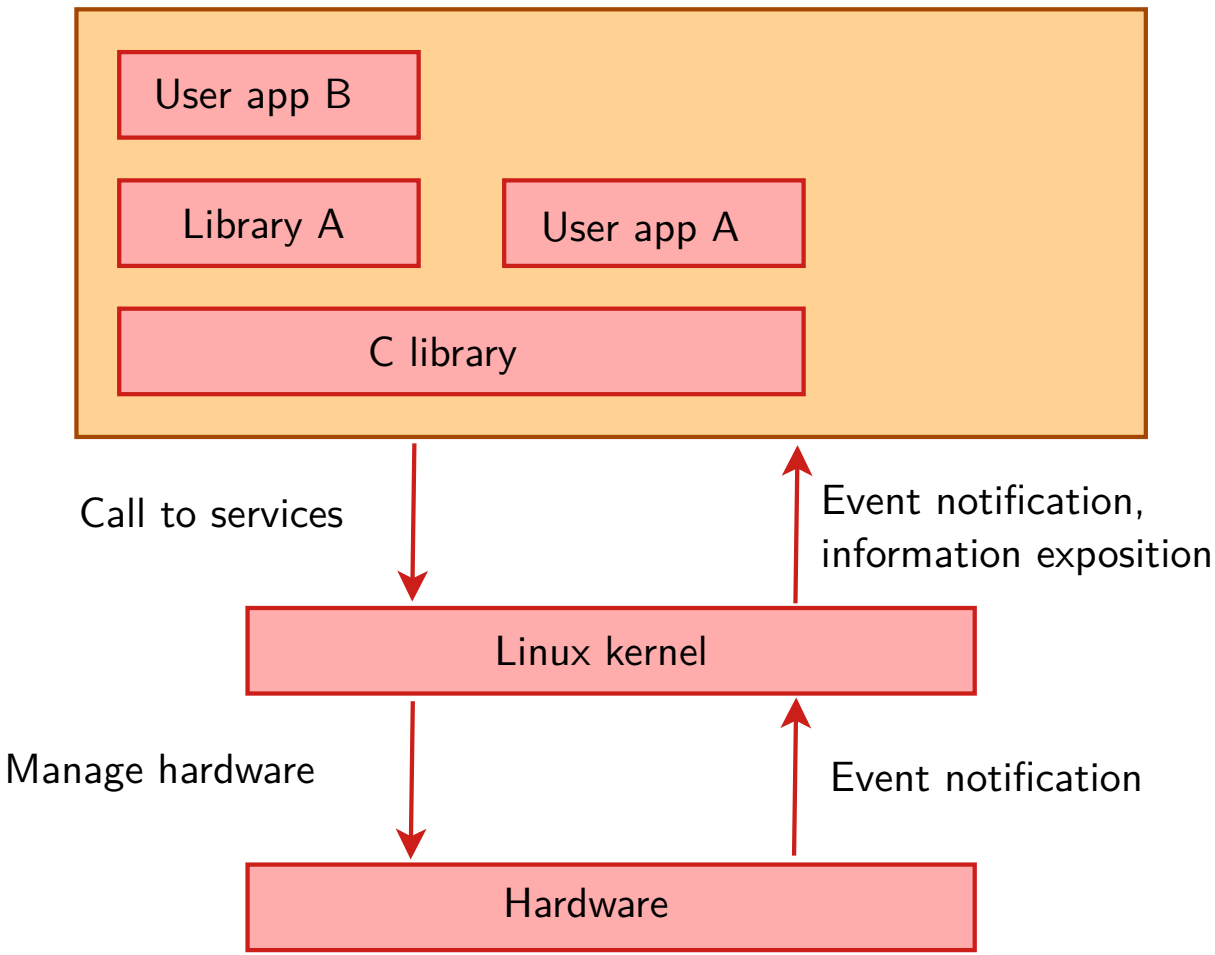
- The Linux kernel was created as a hobby in 1991 by a Finnish student, Linus Torvalds.
  - Linux quickly started to be used as the kernel for free software operating systems
- Linus Torvalds has been able to create a large and dynamic developer and user community around Linux.
- As of today, about 2000+ people contribute to each kernel release, individuals or companies big and small.



Linus Torvalds in 2014
Image credits (Wikipedia): https://bit.ly/2UIa1TD



Linux kernel in the system





Linux kernel main roles

- **Manage all the hardware resources**: CPU, memory, I/O.
- Provide a **set of portable, architecture and hardware independent APIs** to allow user space applications and libraries to use the hardware resources.
- **Handle concurrent accesses and usage** of hardware resources from different applications.
  - Example: a single network interface is used by multiple user space applications through various network connections. The kernel is responsible for "multiplexing" the hardware resource.

System calls

- The main interface between the kernel and user space is the set of system calls
- About 400 system calls that provide the main kernel services
  - File and device operations, networking operations, inter-process communication, process management, memory mapping, timers, threads, synchronization primitives, etc.
- This system call interface is wrapped by the C library, and user space applications usually never make a system call directly but rather use the corresponding C library function

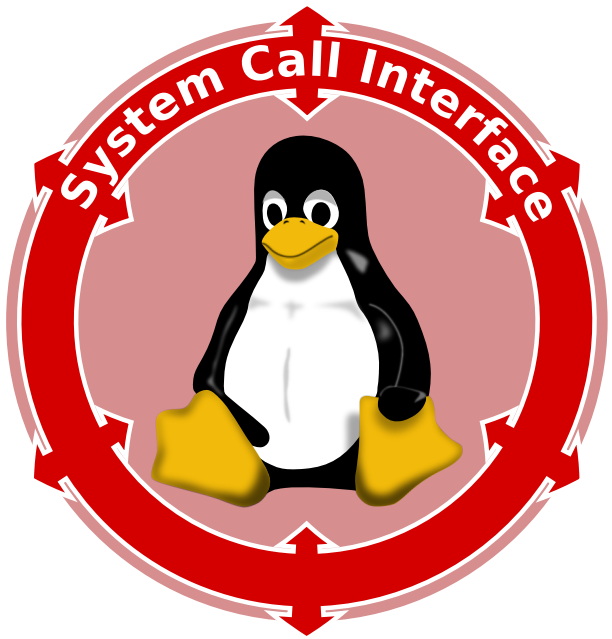


Image credits (Wikipedia): https://bit.ly/2U2rdGB



Pseudo filesystems

- Linux makes system and kernel information available in user space through **pseudo filesystems**, sometimes also called **virtual filesystems**
- Pseudo filesystems allow applications to see directories and files that do not exist on any real storage: they are created and updated on the fly by the kernel
- The two most important pseudo filesystems are
  - `proc`, usually mounted on `/proc`:
    Operating system related information (processes, memory management parameters...)
  - `sysfs`, usually mounted on `/sys`:
    Representation of the system as a tree of devices connected by buses. Information gathered by the kernel frameworks managing these devices.



Linux kernel sources



Location of the official kernel sources

- The mainline versions of the Linux kernel, as released by Torvalds
  - These versions follow the development model of the kernel (`master` branch)
  - They may not contain the latest developments from a specific area yet
  - A good pick for products development phase
  - https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git



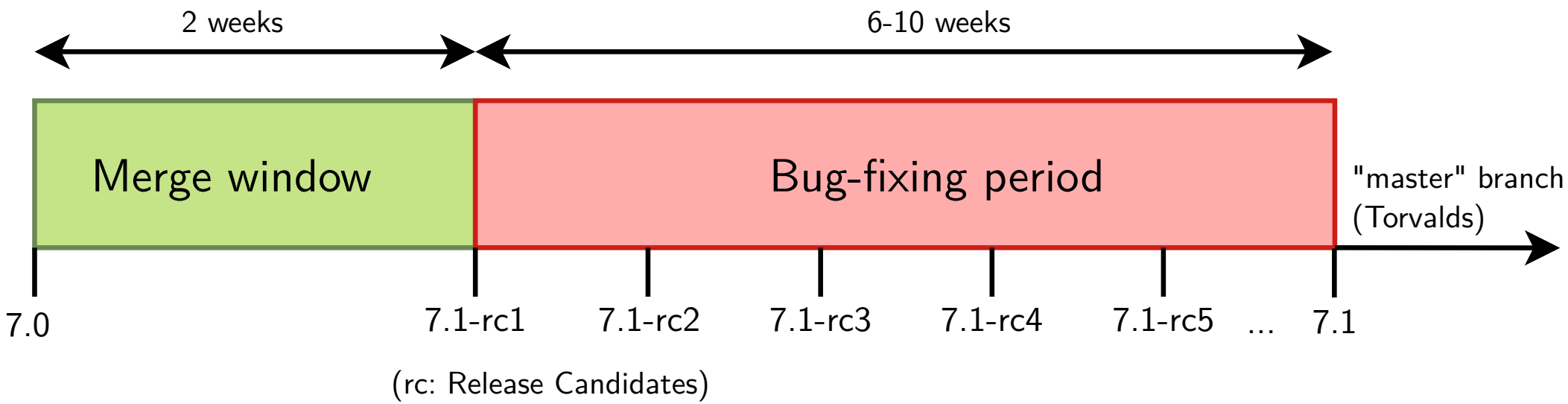
Linux versioning scheme

- Until 2003, there was a new "stabilized" release branch of Linux every 2 or 3 years (2.0, 2.2, 2.4). Development branches took 2-3 years to be merged (too slow!).
- Since 2003, there is a new official release of Linux about every 10 weeks:
  - Versions `2.6` (Dec. 2003) to `2.6.39` (May 2011)
  - Versions `3.0` (Jul. 2011) to `3.19` (Feb. 2015)
  - Versions `4.0` (Apr. 2015) to `4.20` (Dec. 2018)
  - Versions `5.0` (Mar. 2019) to `5.19` (July 2022)
  - Versions `6.0` (Oct. 2022) to `6.19` (Feb. 2026)
  - Versions `7.0` (Apr. 2026) to `7.??`
- Features are added to the kernel in a progressive way. Since 2003, kernel developers have managed to do so without having to introduce a massively incompatible development branch.
- Major updates have no specific meaning!



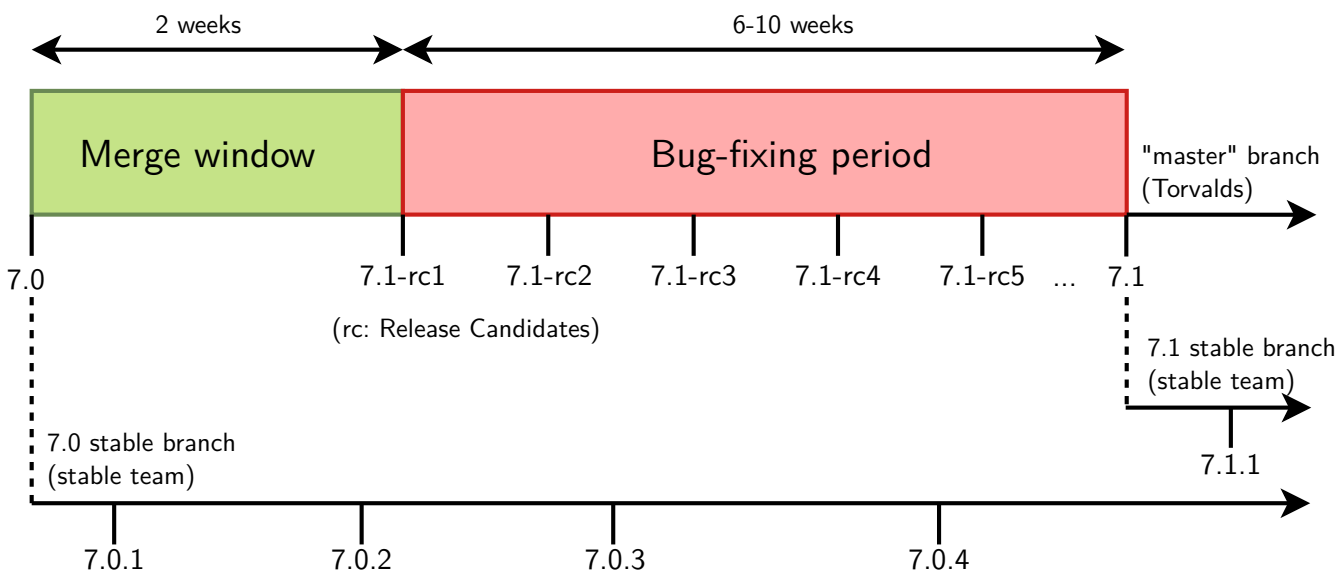
Linux development model

- Each new release starts with a two-week merge window for new features
- Follow about 8 release candidates (one week each)
- Until adoption of a new official release.



Need to further stabilize the official kernels

- Issue: bug and security fixes only merged into the master branch, need to update to the latest kernel to benefit from them.
- Solution: a stable maintainers team goes through all the patches merged into Torvald's tree and backports the relevant ones into their stable branches for at least a few months.





Location of the stable kernel sources

- The stable versions of the Linux kernel, as maintained by a maintainers group
  - These versions do not bring new features compared to Linus' tree
  - Only bug fixes and security fixes are pulled there
  - Each version is stabilized during the development period of the next mainline kernel
  - A good pick for products commercialization phase
  - https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
  - Certain versions will be maintained much longer



Need for long term support

- Issue: bug and security fixes only released for most recent kernel versions.
- Solution: the last release of each year is made an LTS *(Long Term Support)* release, and is supposed to be supported (and receive bug and security fixes) for at least 2 years. These projected EOL may be extended up to 6 years based on the industry interest at large.

Version	Maintainer	Released	Projected EOL
6.18	Greg Kroah-Hartman & Sasha Levin	2025-11-30	Dec, 2027
6.12	Greg Kroah-Hartman & Sasha Levin	2024-11-17	Dec, 2026
6.6	Greg Kroah-Hartman & Sasha Levin	2023-10-29	Dec, 2026
6.1	Greg Kroah-Hartman & Sasha Levin	2022-12-11	Dec, 2027
5.15	Greg Kroah-Hartman & Sasha Levin	2021-10-31	Dec, 2026
5.10	Greg Kroah-Hartman & Sasha Levin	2020-12-13	Dec, 2026

Captured on https://kernel.org in Feb. 2026, following the *Releases* link.

- Example at Google: starting from *Android O (2017)*, all new Android devices have to run such an LTS kernel.



Need for even longer term support

- You could also get long term support from a commercial embedded Linux provider.
  - Wind River Linux can be supported for up to 15 years.
  - Ubuntu Core can be supported for up to 10 years.
- *"If you are not using a supported distribution kernel, or a stable / longterm kernel, you have an insecure kernel"* - Greg KH, 2019
  Some vulnerabilities are fixed in stable without ever getting a CVE.
- The *Civil Infrastructure Platform* project is an industry / Linux Foundation effort to support much longer (at least 10 years) selected LTS versions (currently 4.4, 4.19, 5.10, 6.1 and 6.12) on selected architectures. See
  https://cip-project.org/.



Location of non-official kernel sources

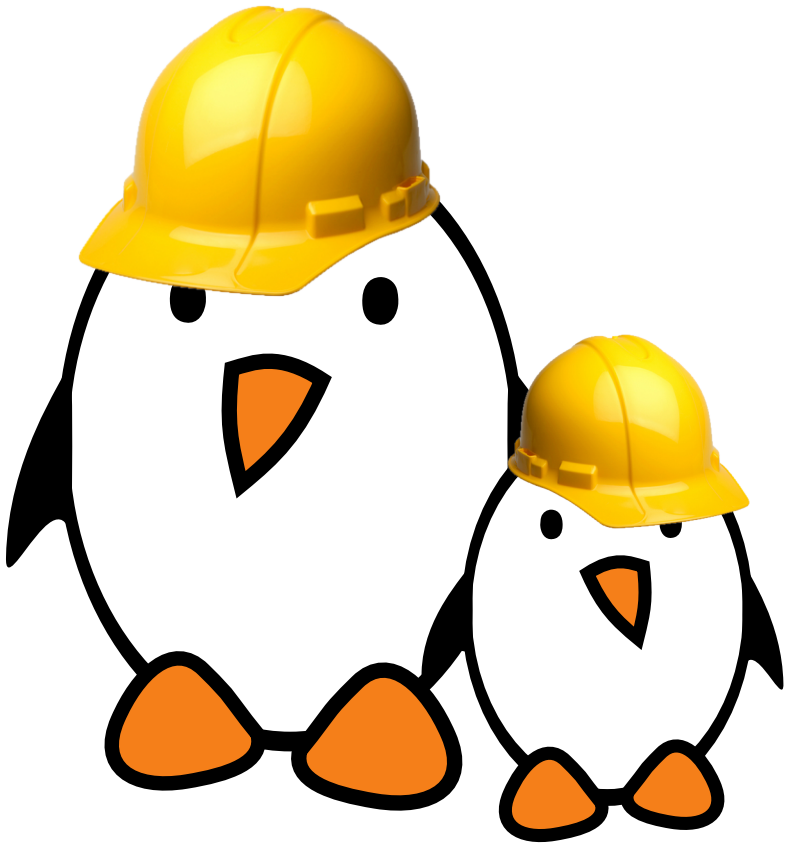
- Many chip vendors supply their own kernel sources
  - Focusing on hardware support first
  - Can have a very important delta with mainline Linux
  - Sometimes they break support for other platforms/devices without caring
  - Useful in early phases only when mainline hasn't caught up yet (many vendors invest in the mainline kernel at the same time)
  - Suitable for PoC, not suitable for products on the long term as usually no updates are provided to these kernels
  - Getting stuck with a deprecated system with broken software that cannot be updated has a real cost in the end
- Many kernel sub-communities maintain their own kernel, with usually newer but fewer stable features, only for cutting-edge development
  - Architecture communities (ARM, MIPS, PowerPC, etc)
  - Device drivers communities (I2C, SPI, USB, PCI, network, etc)
  - Other communities (filesystems, memory-management, scheduling, etc)
  - Not suitable to be used in products



Linux kernel size and structure

- Linux v6.19 sources: close to 92k files, 42M lines, 1.4GiB
- But a compressed Linux kernel just sizes a few megabytes.
- So, why are these sources so big? Because they include numerous device drivers, network protocols, architectures, filesystems... The core is pretty small!
- As of kernel version v6.19 (in percentage of total number of lines):
  - drivers/: 61%
  - arch/: 12%
  - tools/: 5.4%
  - sound/: 3.9%
  - fs/: 3.8%
  - Documentation/: 3.8%
  - include/: 3.4%
  - net/: 3.1%
  - kernel/: 1.3%
  - lib/: 0.8%
  - mm/: 0.5%
  - scripts/: 0.3%
  - security/, rust/, crypto/, block/, samples/, io_uring/, virt/, usr/, LICENSES/, ipc/, init/, certs/: ≤ 0.3%
  - Build system files: Kbuild, Kconfig, Makefile
  - Other files: COPYING, CREDITS, MAINTAINERS, README

Practical lab - Downloading kernel source code



- Clone the mainline Linux source tree with git



Linux kernel source code



Programming language

- Implemented in C like all UNIX systems
- A little Assembly is used too:
  - CPU and machine initialization, exceptions
  - Critical library routines.
- No C++ used, see https://lkml.org/lkml/2004/1/20/20
- Rust support is currently being introduced: drivers/net/phy/ax88796b_rust.rs is a first driver written in Rust.
- All the code compiled with gcc
  - Many gcc specific extensions used in the kernel code, any ANSI C compiler will not compile the kernel
  - See https://gcc.gnu.org/onlinedocs/gcc-10.2.0/gcc/C-Extensions.html
- A subset of the supported architectures can be built with the LLVM C compiler (Clang) too: https://clangbuiltlinux.github.io/



No C library

- The kernel has to be standalone and can't use user space code.
- Architectural reason: user space is implemented on top of kernel services, not the opposite.
- Technical reason: the kernel is on its own during the boot up phase, before it has accessed a root filesystem.
- Hence, kernel code has to supply its own library implementations (string utilities, cryptography, uncompression...)
- So, you can't use standard C library functions in kernel code (`printf()`, `memset()`, `malloc()`,...).
- Fortunately, the kernel provides similar C functions for your convenience, like printk(), memset(), kmalloc(), ...



Portability

- The Linux kernel code is designed to be portable
- All code outside arch/ should be portable
- To this aim, the kernel provides macros and functions to abstract the architecture specific details
  - Endianness
  - I/O memory access
  - Memory barriers to provide ordering guarantees if needed
  - DMA API to flush and invalidate caches if needed
- Never use floating point numbers in kernel code. Your code may need to run on a low-end processor without a floating point unit.

Linux kernel to user API/ABI stability

Linux kernel to userspace API is stable

- Source code for userspace applications will not have to be updated when compiling for a more recent kernel
  - System calls, `/proc` and `/sys` content cannot be removed or changed. Only new entries can be added.

Linux kernel to userspace ABI is stable

- Binaries are portable and can be executed on a more recent kernel
  - The way memory is accessed, the size of the variables in memory, how structures are organized, the calling convention, etc, are all stable over time.

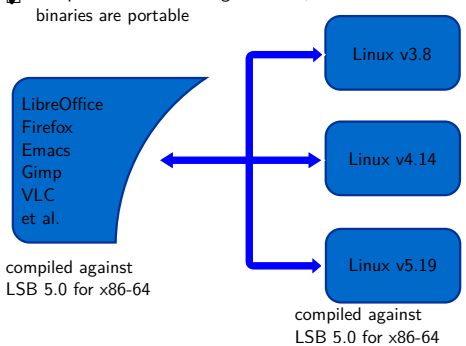
Linux kernel to user API

☑ API stability **is** guaranteed, source code is portable!



Linux kernel to user ABI

☑ compatible ABI **can be** guaranteed, binaries are portable



Modified Image from Wikipedia:
https://bit.ly/2U2rdGB



Linux internal API/ABI instability

Linux internal API is not stable

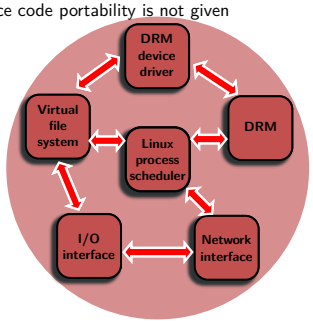
- The source code of a driver is not portable across versions
  - In-tree drivers are updated by the developer proposing the API change: works great for mainline code
  - An out-of-tree driver compiled for a given version may no longer compile or work on a more recent one
  - See process/stable-api-nonsense for reasons why

Linux internal ABI is not stable

- A binary module compiled for a given kernel version cannot be used with another version
  - The module loading utilities will perform this check prior to the insertion

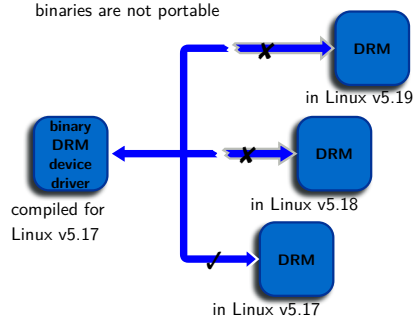
Linux internal API

✘ API stability **is not** guaranteed, source code portability is not given



Linux internal ABI

✘ **no** stable ABI over Linux kernel releases, binaries are not portable



Modified Image from Wikipedia: https://bit.ly/2U2rdGB



Kernel memory constraints

- No memory protection
- The kernel doesn't try to recover from attempts to access illegal memory locations. It just dumps *oops* messages on the system console.
- Fixed size stack (8 or 4 KB). Unlike in user space, no mechanism was implemented to make it grow. Don't use recursion!
- Swapping is not implemented for kernel memory either
  (except *tmpfs* which lives completely in the page cache and on swap)



Linux kernel licensing constraints

- The Linux kernel is licensed under the GNU General Public License version 2
  - This license gives you the right to use, study, modify and share the software freely
- However, when the software is redistributed, either modified or unmodified, the GPL requires that you redistribute the software under the same license, with the source code
  - If modifications are made to the Linux kernel (for example to adapt it to your hardware), it is a derivative work of the kernel, and therefore must be released under GPLv2.
- The GPL license has been successfully enforced in courts: https://en.wikipedia.org/wiki/Gpl-violations.org#Notable_victories
- However, you're only required to do so
  - At the time the device starts to be distributed
  - To your customers, not to the entire world



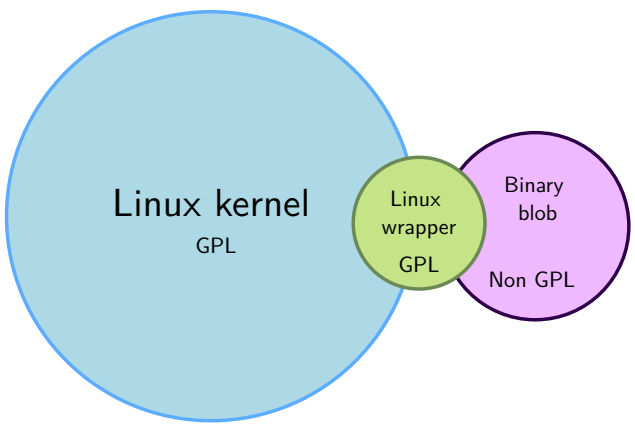
Proprietary code and the kernel

- It is illegal to distribute a binary kernel that includes statically compiled proprietary drivers
- The kernel modules are a gray area: unclear if they are legal or not
  - The general opinion of the kernel community is that proprietary modules are bad: process/kernel-driver-statement
  - From a legal point of view, each driver is probably a different case:
    - Are they derived works of the kernel?
    - Are they designed to be used with another operating system?
- Is it really useful to keep drivers secret anyway?

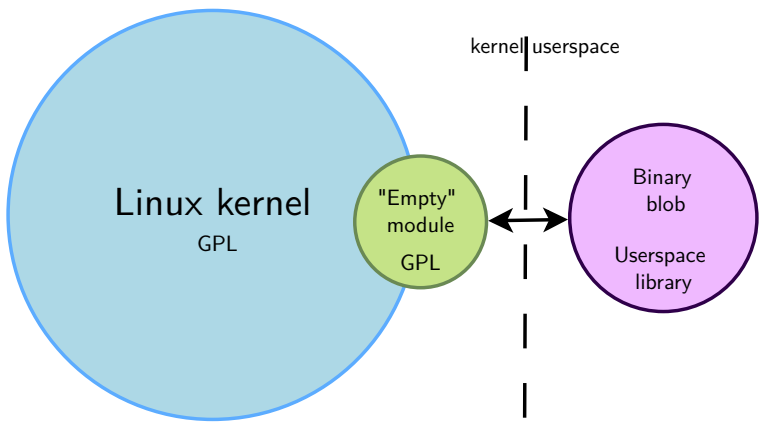


Abusing the kernel licensing constraints

- There are some examples of proprietary drivers
  - Nvidia uses a wrapper between their drivers and the kernel
  - They claim the drivers could be used with a different OS with another wrapper
  - Unclear whether it makes it legal or not



- The current trend is to hide the logic in the firmware or in userspace. The GPL kernel driver is almost empty and either:
  - Blindly writes an incoming flow of bytes in the hardware
  - Exposes a huge MMIO region to userspace through `mmap`





Advantages of GPL drivers

- You don't have to write your driver from scratch. You can reuse code from similar free software drivers.
- Your drivers can be freely and easily shipped by others (for example by Linux distributions or embedded Linux build systems).
- Legal certainty, you are sure that a GPL driver is fine from a legal point of view.



Advantages of mainlining your kernel drivers

- The community, reviewers and maintainers will review your code before accepting it, offering you the opportunity to enhance it and understand better the internal APIs.
- Once accepted, you will get cost-free bug and security fixes, support for new features, and general improvements.
- Your work will automatically follow the API changes.
- Accessing your code will be much easier for users.
- Your code will remain valid no matter the kernel version.

This will for sure reduce your maintenance and support work



User space device drivers 1/2

- The kernel provides some mechanisms to access hardware from userspace:
  - USB devices with *libusb*, https://libusb.info/
  - SPI devices with *spidev*, spi/spidev
  - I2C devices with *i2cdev*, i2c/dev-interface
  - GPIOs with *libgpiod*, https://libgpiod.readthedocs.io
  - Memory-mapped devices with *UIO*, including interrupt handling, driver-api/uio-howto
- These solutions can only be used if:
  - There is no need to leverage an existing kernel subsystem such as the networking stack or filesystems.
  - There is no need for the kernel to act as a "multiplexer" for the device: only one application accesses the device.
- Certain classes of devices like printers and scanners do not have any kernel support, they have always been handled in user space for historical reasons.
- Otherwise this is ***not*** how the system should be architectured. Kernel drivers should always be preferred!



User space device drivers 2/2

- Advantages
  - No need for kernel coding skills.
  - Drivers can be written in any language, even Perl!
  - Drivers can be kept proprietary.
  - Driver code can be killed and debugged. Cannot crash the kernel.
  - Can use floating-point computation.
  - Potentially higher performance, especially for memory-mapped devices, thanks to the avoidance of system calls.
- Drawbacks
  - The kernel has no longer access to the device.
  - None of the standard applications will be able to use it.
  - Cannot use any hardware abstraction or software helpers from the kernel
  - Need to adapt applications when changing the hardware.
  - Less straightforward to handle interrupts: increased latency.

Practical lab - Kernel Source Code - Exploring



- Explore kernel sources manually
- Use automated tools to explore the source code